Bhartiya Vidya Bhavans
Sardar Patel Institute of Technology
Munshi Nagar – Andheri West – 400058

Name: Atharv patil (2024300165)
Tanish parakh (2024300157)
Gitesh patil (2024300166)
Branch: C.E

DIVISION: C
BATCH: C1
DATE: 24/08/2026

Experiment No.: 4

Aim : To implement and evaluate The Bully Algorithm for Leader / Coordinator Election in a Distributed Banking Home Loan Management System, demonstrating dynamic election of the highest active process ID as the Central Master Coordinator upon coordinator failure or recovery.

Theory :

In a distributed system, a Coordinator Node (Leader) is responsible for central management, centralized locking, transaction approval, and resource allocation. If the coordinator process crashes or fails, the system must dynamically run an Election Algorithm to choose a new coordinator among active processes.

1. The Bully Algorithm (Garcia-Molina):
The Bully Algorithm assumes that every process has a unique, globally known Process ID (priority). The process with the highest active Process ID is always elected as the Coordinator ('bullying' lower ID processes).
Algorithm Steps:
• **Election Initiation:** When process P_i detects that the current coordinator is unresponsive, it sends an ELECTION message to all processes with higher IDs (P_{i+1}, ..., P_N).
• **Response (OK / ANSWER):** If a higher process P_k is alive, it sends an OK message back to P_i and starts its own election.
• **Timeout & Victory:** If no higher process responds within a timeout period T, process P_i declares itself the winner and broadcasts a COORDINATOR message to all lower active processes.
• **Process Recovery:** If a crashed process with a higher ID comes back online, it immediately initiates an election and reclaims the coordinator role.

2. Message Complexity Analysis:
• **Best-Case Complexity:** O(N) — When the process with the second-highest ID detects failure.
• **Worst-Case Complexity:** O(N^2) — When the process with the lowest ID detects failure.

Key Concepts Demonstrated:
● **Leader Election:** Dynamic coordinator selection without human intervention.
● **High Availability:** Guarantees distributed banking system continuity upon node crashes.
● **Bully Behavior:** Highest priority active process always reclaims control.

Problem-solving :

Code :

1. BullyElectionBanking.java (The Bully Algorithm Implementation)

```
import java.util.*;

public class BullyElectionBanking {

    // Process class representing a distributed banking node
    static class BankNode {
        int id;
        String name;
        boolean isAlive;
        boolean isCoordinator;

        BankNode(int id, String name) {
            this.id = id;
            this.name = name;
            this.isAlive = true;
            this.isCoordinator = false;
        }

        @Override
        public String toString() {
            return name + " (ID: " + id + ")";
        }
    }

    private static Map<Integer, BankNode> cluster = new HashMap<>();
    private static int coordinatorId = 4; // Initial highest ID process

    public static void main(String[] args) throws InterruptedException {
        System.out.println("================================================================================");
        System.out.println(" EXPERIMENT 4: BULLY ELECTION ALGORITHM IN DISTRIBUTED BANKING SYSTEM");
        System.out.println("================================================================================");

        // Initialize Banking Cluster Nodes
        cluster.put(1, new BankNode(1, "BankNode-1 [Branch Mumbai]"));
        cluster.put(2, new BankNode(2, "BankNode-2 [Branch Delhi]"));
        cluster.put(3, new BankNode(3, "BankNode-3 [Branch Bangalore]"));
        cluster.put(4, new BankNode(4, "BankNode-4 [Central Master Server]"));

        cluster.get(coordinatorId).isCoordinator = true;

        System.out.println("\n--- Step 1: Initial Cluster State ---");
        displayClusterState();
        System.out.println("Current Active Coordinator: " + cluster.get(coordinatorId));

        // --------------------------------------------------------------------
        // SCENARIO 1: Coordinator (BankNode-4) Crashes
        // --------------------------------------------------------------------
        System.out.println("\n--- Step 2: Central Master Coordinator (BankNode-4) Crashes ---");
        cluster.get(4).isAlive = false;
        cluster.get(4).isCoordinator = false;
        System.out.println("[CRASH EVENT] " + cluster.get(4).name + " has crashed/gone offline!");

        // Node 2 detects coordinator failure and initiates election
        System.out.println("\n--- Step 3: Node 2 Detects Failure & Initiates Bully Election ---");
        initiateElection(2);

        // --------------------------------------------------------------------
        // SCENARIO 2: Higher Node (BankNode-4) Recovers & Claims Leadership ("Bully")
        // --------------------------------------------------------------------
        System.out.println("\n--- Step 4: Crashed Node (BankNode-4) Recovers & Claims Leadership ---");
        System.out.println("[RECOVERY EVENT] " + cluster.get(4).name + " is coming back online...");
        cluster.get(4).isAlive = true;

        // Recovered node 4 initiates election to reclaim coordinator role
        initiateElection(4);

        System.out.println("\n================================================================================");
        System.out.println(" FINAL CLUSTER STATE AFTER ELECTION & RECOVERY:");
        displayClusterState();
        System.out.println("================================================================================");
    }
```

```
    private static void initiateElection(int initiatorId) {
        BankNode initiator = cluster.get(initiatorId);
        System.out.println("\n>>> [ELECTION STARTED] Initiated by " + initiator);

        boolean higherNodeResponded = false;

        // Send ELECTION message to all processes with higher IDs
        for (int id = initiatorId + 1; id <= 4; id++) {
            BankNode target = cluster.get(id);
            System.out.println("  [SEND ELECTION] " + initiator.name + " ---> ELECTION ---> " + target.name);

            if (target.isAlive) {
                System.out.println("  [RECEIVE OK]    " + target.name + " ---> OK / ANSWER ---> " + initiator.nam
e);
                higherNodeResponded = true;
            } else {
                System.out.println("  [TIMEOUT]       No response from " + target.name + " (Node is DOWN)");
            }
        }

        if (!higherNodeResponded) {
            // No higher node responded, initiator becomes the new Coordinator
            setNewCoordinator(initiatorId);
        } else {
            // Higher nodes responded, pass election responsibility to the highest alive node
            for (int id = initiatorId + 1; id <= 4; id++) {
                if (cluster.get(id).isAlive) {
                    initiateElection(id);
                    break;
                }
            }
        }
    }

    private static void setNewCoordinator(int newCoordId) {
        coordinatorId = newCoordId;
        for (BankNode node : cluster.values()) {
            node.isCoordinator = (node.id == newCoordId);
        }

        BankNode newCoord = cluster.get(newCoordId);
        System.out.println("\n==========================================================================");
        System.out.println(" [ELECTION RESULT] " + newCoord.name + " WON THE ELECTION!");
        System.out.println("==========================================================================");

        // Broadcast COORDINATOR message to all lower active processes
        for (BankNode node : cluster.values()) {
            if (node.id != newCoordId && node.isAlive) {
                System.out.println("  [BROADCAST COORDINATOR] " + newCoord.name + " ---> COORDINATOR Announcement
 ---> " + node.name);
            }
        }
    }

    private static void displayClusterState() {
        for (BankNode node : cluster.values()) {
            String state = node.isAlive ? (node.isCoordinator ? "COORDINATOR (MASTER)" : "ACTIVE (SLAVE)") : "CRA
SHED (DOWN)";
            System.out.printf("  - %-42s : State = %s\n", node, state);
        }
    }
}
```

Output screenshot :

Figure 1: Initial Cluster State with BankNode-4 as Active Coordinator

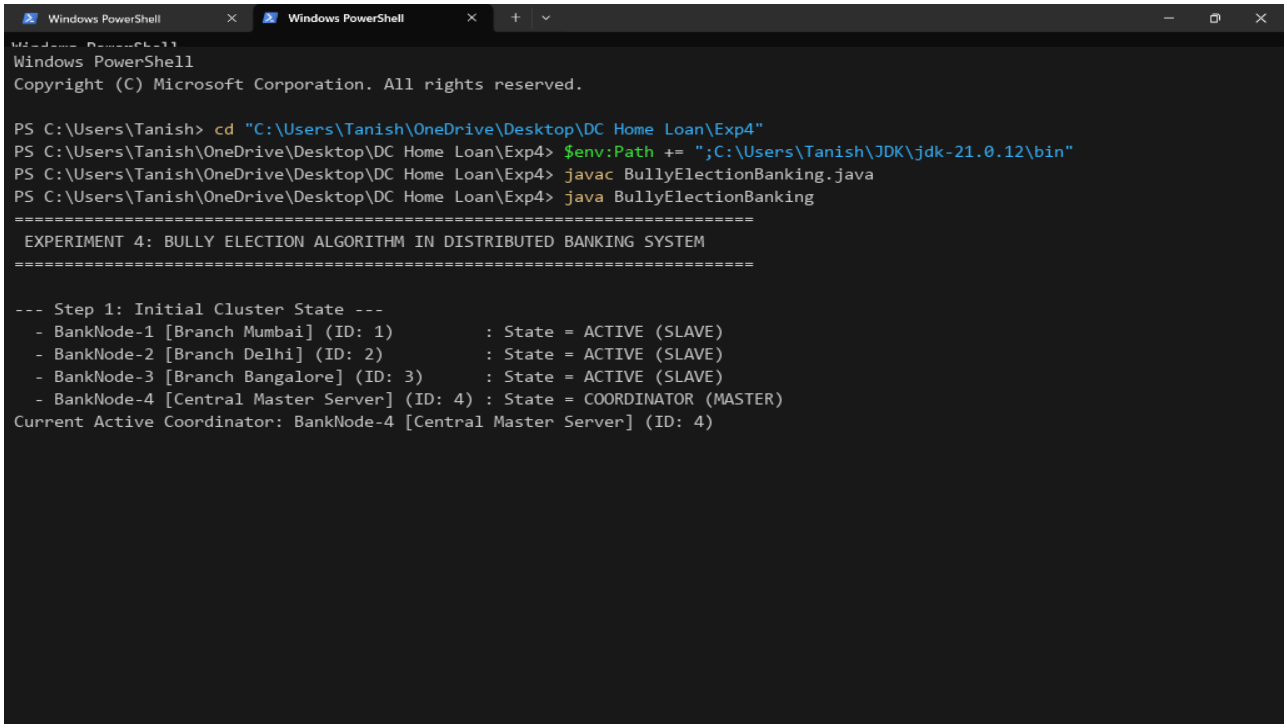


Figure 2: Central Master Coordinator Crash Event & Election Initiation by BankNode-2

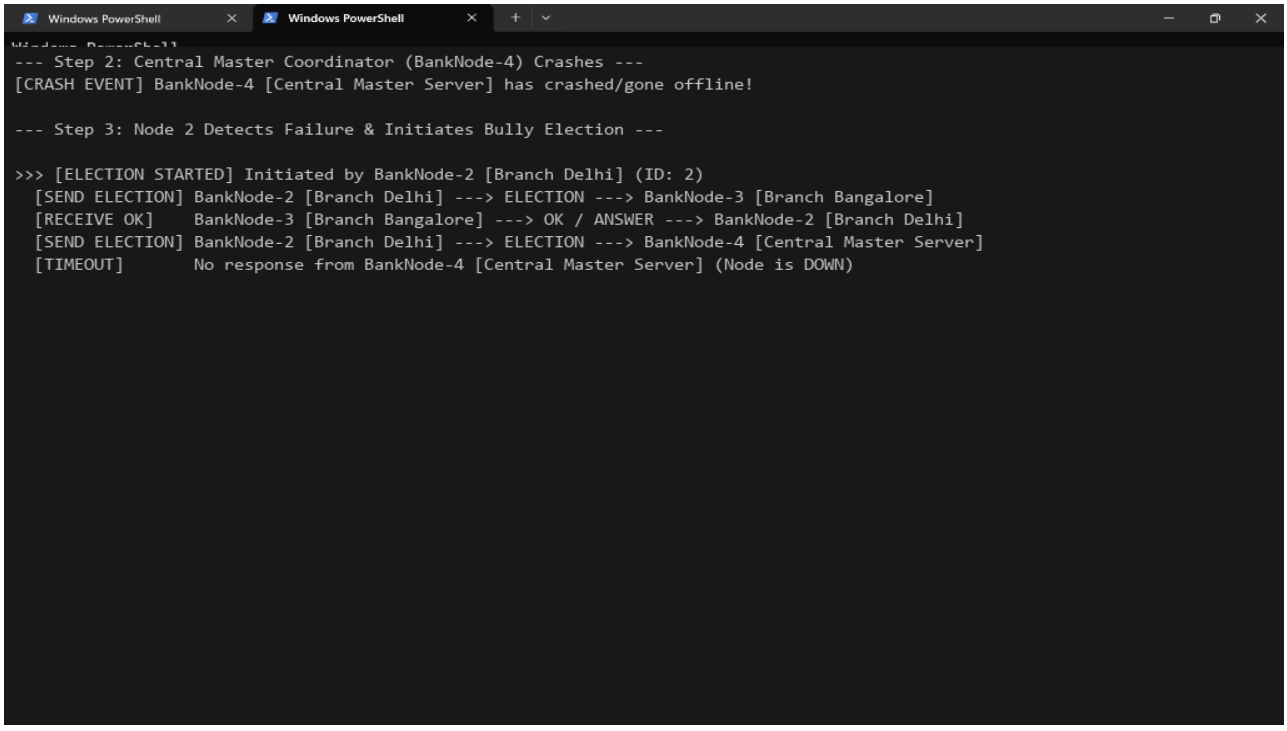


Figure 3: BankNode-3 Winning Election & Broadcasting COORDINATOR Announcement

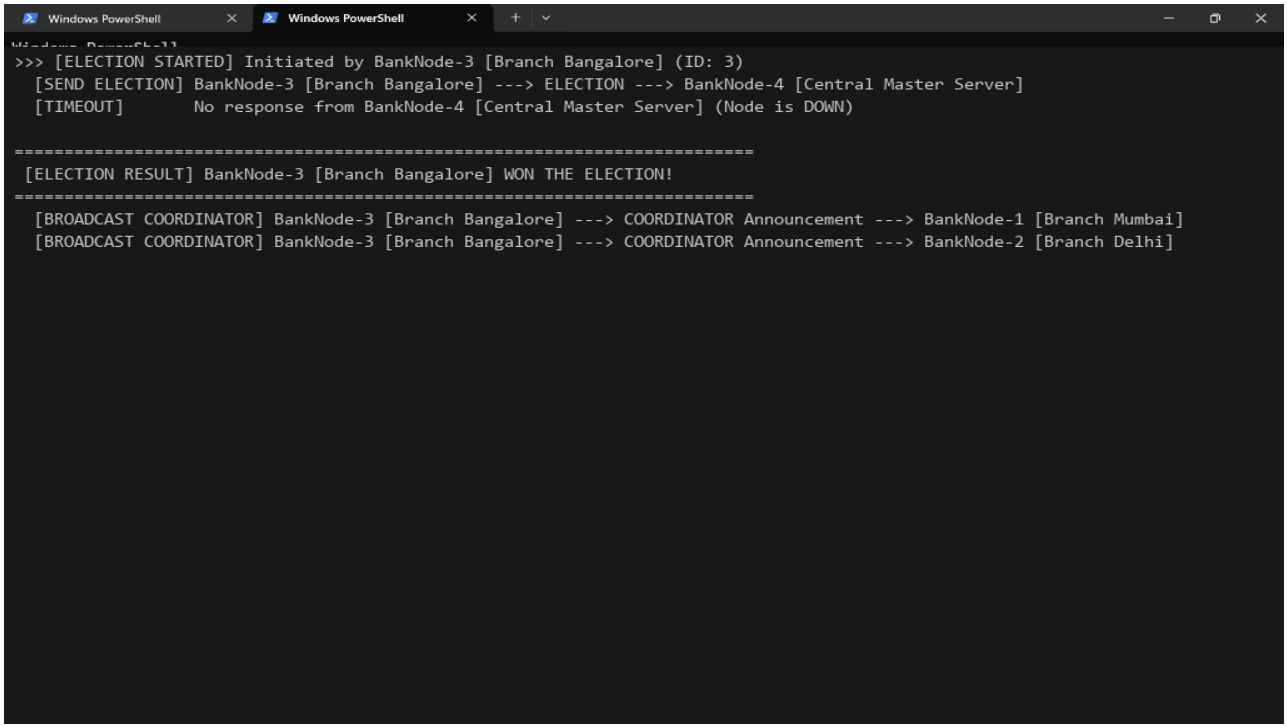
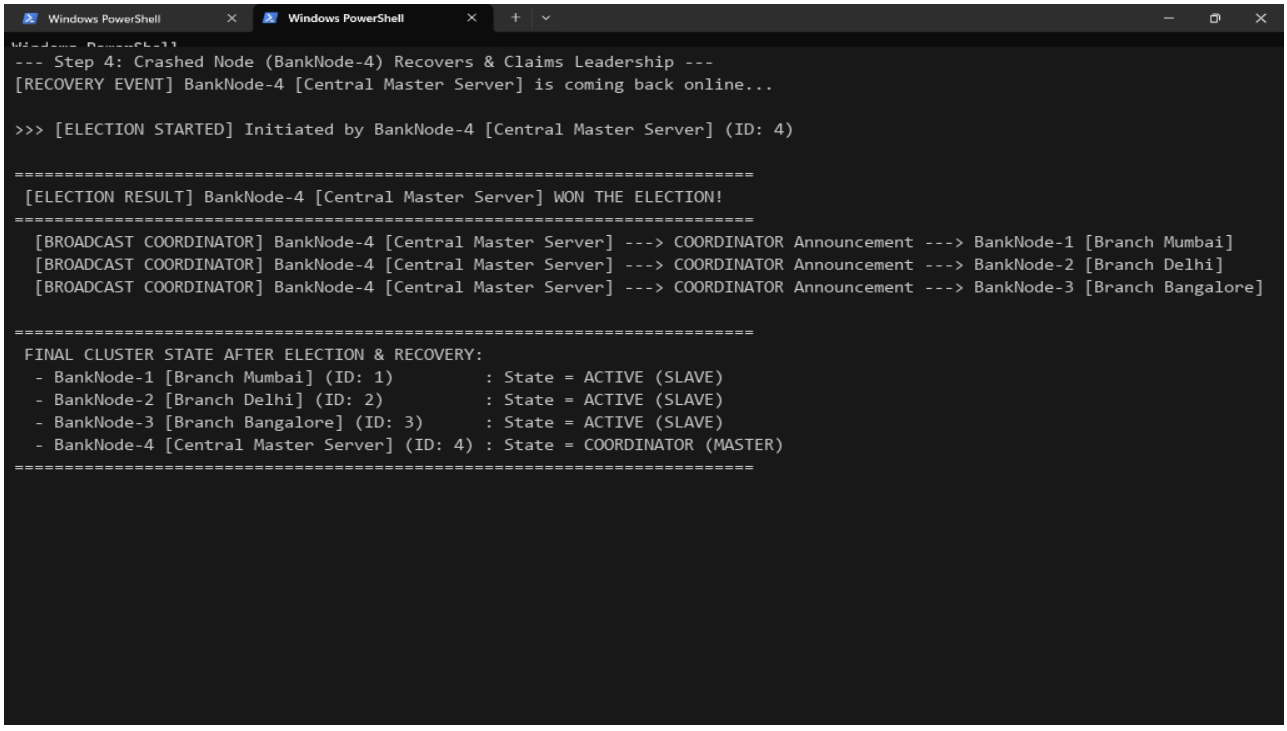


Figure 4: BankNode-4 Recovery & Reclaiming Leadership as Central Master Coordinator



Conclusion: (It should include what the student has learned/ understood from the experiment).

In this experiment, we successfully implemented and analyzed The Bully Algorithm for Leader / Coordinator Election in a Distributed Banking System. We demonstrated how active banking nodes handle coordinator failure by initiating election messages, exchanging OK responses, and electing the highest active Process ID (BankNode-3) as the new master. Furthermore, we showed how a recovered node (BankNode-4) initiates an election to reclaim its leadership position. This experiment effectively highlighted the fault-tolerance and high-availability properties of election algorithms in distributed systems.

Attach research paper based on experiment performed into the classroom (Not for all the practicals)

(Students are required to prepare one doc file for each experiment and then upload it in pdf form.)